

CESE 4025 - Real-Time Systems: Assignment A, Group: 02

Zubair Al-Zubi (5342007), Mustafa Yücesan (5655544)

December 7, 2024

Report contains 6 of maximum 6 pages.

1 Introduction

This assignment focuses on the implementation and analysis of real-time scheduling algorithms for embedded systems using the STM32F4DISCOVERY platform.

2 System specifications

1. The example scheduler is not a real-time scheduler. A real-time scheduler takes deadlines into account, and this example scheduler doesn't. It just executes tasks one by one.
2. The periods of the tasks of **TASK_SET_1** are 20, 40 and 60 ms. The hyper-period is then the LCM (least common multiple) of (20, 40, 60) is 120 ms. The utilization factor of **TASK_SET_1** is: $2/20 + 10/40 + 30/60 = 0.85$.
3. The goal here is to design two non-trivial task sets with at least 3 tasks and a maximum hyper-period of 120 ms. **TASK_SET_2** should have a utilization factor of 1 ($U = 1$), and **TASK_SET_3** should have a utilization factor greater than 1 ($U > 1$). The utilization factor (U) is defined as $U = \sum_{i=1}^n C_i/T_i$ [1], where C_i is the execution time and T_i is the period [1].
 - (a) Let $T_1 = 40$, $T_2 = 60$, $T_3 = 120$ ms. Then, the hyper-period of **TASK_SET_2** is $\text{LCM}(40, 60, 120) = 120$ ms. Now to determine the execution times (C_i):

$$\frac{C_1}{40} + \frac{C_2}{60} + \frac{C_3}{120} = 1 \iff 3C_1 + 2C_2 + C_3 = 120. \quad (1)$$

A possible solution for C_i is: $C_3 = 12$ and $C_2 = 24$ which gives $C_1 = 20$ ms. Thus, a possible design for **TASK_SET_2** is:

```
spawn_task(20, 40, leds[0]);  
spawn_task(24, 60, leds[1]);  
spawn_task(12, 120, leds[2]);
```

The processor utilization factor is $U = 20/40 + 24/60 + 12/120 = 1$, as expected.

- (b) Similarly, for $U > 1$: Let $T_1 = 20$, $T_2 = 30$, $T_3 = 40$ ms. Then, the hyper-period of **TASK_SET_3** is $\text{LCM}(20, 30, 40) = 120$ ms.

$$\frac{C_1}{20} + \frac{C_2}{30} + \frac{C_3}{40} > 1 \iff 6C_1 + 4C_2 + 3C_3 > 120. \quad (2)$$

Statement on the use of generative AI* (tick what applies, multiple options are possible):

When doing this assignment, I/we:

0 Did not use generative AI.

0 Used generative AI to generate (parts of) the code. Where applicable, I/we have explained this in a comment.

0 Used generative AI to generate (parts of) the report. Where applicable, I/we have explained this in a footnote.

1 Used generative AI to lint/format the code and/or improve the grammar, style, spelling of the report.

A possible solution for C_i is: $C_3 = 20$ and $C_2 = 15$ which gives $C_1 = 10$ ms. Thus, a possible design for TASK_SET_3 is:

```
spawn_task(10, 20, leds[0]);
spawn_task(15, 30, leds[1]);
spawn_task(20, 40, leds[2]);
```

The processor utilization factor is $U = 10/20 + 15/30 + 20/40 = 1.5$, which is greater than 1.

3 Rate Monotonic Scheduler

3.1 Implementation

A Rate Monotonic (RM) scheduler is implemented to manage task execution based on task periods. In this scheduler, tasks with shorter periods are assigned higher priorities.

The scheduler operates in a continuous loop, monitoring the current time to release tasks at their specified periods. At each scheduling point, it selects the highest-priority ready task (one with remaining execution time) to run. If a higher-priority task becomes ready while a lower-priority task is running, the scheduler preempts the lower-priority task to ensure timely execution of the more critical task.

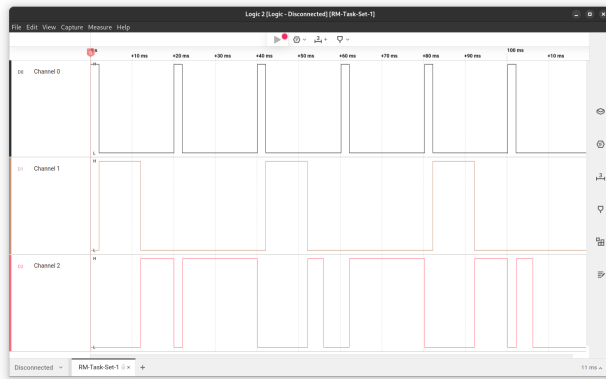
Preemption is handled by updating the remaining execution time of the preempted task so it can resume later. If no tasks are ready to run, the scheduler places the system in an idle state to conserve resources.

The following additional fields have been added to the **Task** struct in **scheduler.h**:

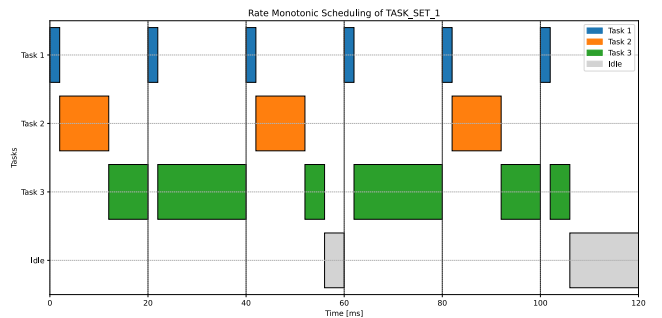
- **next_release_time**: Indicates the next time the task is scheduled to be released.
- **remaining_time**: Tracks the remaining execution time for the task during its current activation.

3.2 Verification

The logic analyzer software *Saleae Logic 2* was used to capture the execution timeline of the tasks. The output of the logic analyzer for Task Set 1 ($U < 1$) is shown in Figure 1a.



(a) Logic analyzer output of RM scheduling of Task Set 1 ($U = 0.85$)



(b) Theoretical RM plot of Task Set 1

Figure 1: Comparison of RM scheduling for Task Set 1: Logic analyzer output vs. Theoretical plot

The figure illustrates the preemptive execution of tasks based on the RM scheduling algorithm, with Task 1 having the highest priority, followed by Task 2 and Task 3.

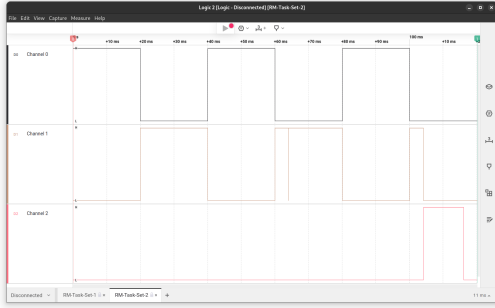
The output was compared to the expected theoretical results shown in Figure 1b, confirming that it aligns with the predicted outcomes.

3.3 Analysis

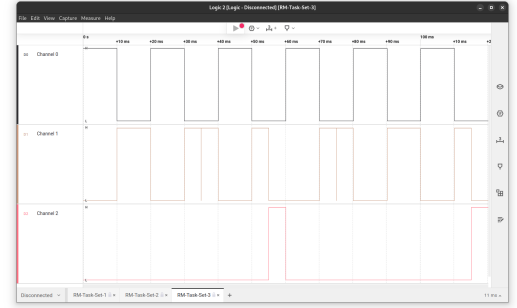
3.3.1 Custom task set

The RM scheduler was tested using the three different aforementioned task sets (**TASK_SET_1**, **TASK_SET_2**, **TASK_SET_3**) with varying utilization factors ($U < 1$, $U = 1$, $U > 1$) to analyze its behavior under different conditions.

1. Task Set 1 ($U < 1$): In this task set, the total processor utilization was less than 1, meaning that the processor had enough time to execute all tasks within their deadlines. The scheduler operates as expected, executing each task within its respective period while respecting priority order (**Figure 1a**). Idle time is observed during periods when no task is ready.
2. Task Set 2 ($U = 1$): Since the total utilization is exactly 1, theoretically, the system should be fully utilized with no idle time. However, idle time was observed. This occurs because the periods of the tasks do not align in a way that continuously fills the CPU's available time. This misalignment creates gaps where no task is ready to run (**Figure 2a**).
3. Task Set 3 ($U > 1$): Here, the total processor utilization exceeded 1, which means that the processor could not execute all tasks within their deadlines. As expected, higher-priority tasks were executed first, but lower-priority tasks missed their deadlines due to the lack of processing time. This resulted in missed deadlines and overall overload of the system.



(a) Logic analyzer output of RM scheduling of Task Set 2 ($U = 1$)



(b) Logic analyzer output of RM scheduling of Task Set 3 ($U = 1.5$)

Figure 2: RM Scheduling Results for Different Task Sets

3.3.2 Overhead

The overhead of the RM scheduler was measured using the given task set in `main.c` (**TASK_SET_1**). Overhead refers to the time consumed by the scheduler to perform its scheduling operations, excluding the actual execution time of the tasks.

When zooming in on Task Set 1, the time difference between the completion of one job and the start of the subsequent job (at the point where the scheduler is actively making decisions) was measured to be approximately 30 μ s, as can be seen in **Figure 3**.

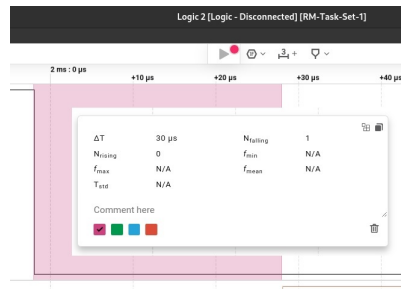


Figure 3: Task Set 1 Time Difference

Thus, the percentage of the hyper-period that is spent on this overhead is:

$$\text{Overhead Percentage} = \frac{\text{Overhead Time}}{\text{Hyper-period}} = \frac{30 \mu\text{s}}{120 \text{ ms}} = 0.025\%. \quad (3)$$

During overhead, the scheduler performs context switching, task selection based on priority, interrupt handling, and updates task states, all contributing to the overhead time [1, 3, 4].

3.3.3 Overloading

Overloading occurs when the total CPU utilization demanded by all tasks exceeds the processor’s capacity, i.e., the total utilization $U > 1$ [1]. This means the processor cannot execute all tasks within their deadlines because there isn’t enough processing time available.

The RM scheduler assumes that if $U \leq 1$, all tasks will meet their deadlines. However, when $U > 1$, the scheduler cannot guarantee deadline adherence.

1. **Task Set 1** ($U < 1$): With a total utilization of 0.85, the RM scheduler efficiently managed the tasks, ensuring that all deadlines were met. Higher-priority tasks preempted lower-priority ones as expected, resulting in optimal CPU utilization with minimal idle time.
2. **Task Set 2** ($U = 1$): Despite the total utilization equaling the CPU capacity, idle time was observed. This occurred because the task periods did not align perfectly, creating gaps where no tasks were ready to execute. Consequently, lower-priority tasks occasionally missed deadlines, highlighting RM scheduler limitations at full utilization.
3. **Task Set 3** ($U > 1$): With a total utilization of 1.5, the RM scheduler prioritized higher-priority tasks, leading to deadline misses for lower-priority tasks. The processor was unable to handle the excessive load, confirming that RM scheduling struggles under overloading conditions.

4 Earliest Deadline First Scheduler

4.1 Implementation

The earliest deadline first (EDF) scheduler selects the task with the nearest absolute deadline to execute next. The function has been implemented in `scheduler.c`. An additional field, `deadline`, has been added to the `Task` struct, which represents the absolute deadline by which the current job of the task must be completed.

The EDF scheduler operates by continuously monitoring the current system time and updating the state of each task based on their `deadline` and `remaining_time`. When a task’s release time arrives, its `remaining_time` is reset to its designated `execution_time`, and its `deadline` is set to the current time plus its `period`. The scheduler then selects the task with the earliest deadline from all the active tasks to be executed. Preemption is managed by comparing the deadlines of the currently executing task and the newly selected task, ensuring that tasks with more nearby deadlines are given priority over others.

4.2 Verification

The EDF scheduler was also verified using the logic analyzer. The output of the given task set (TASK_SET_1) is plotted in [Figure 4a](#).

The tasks were executed as expected, meeting their deadlines. In particular, tasks with earlier deadlines successfully preempted those with later deadlines, verifying that the EDF scheduling strategy was correctly implemented.

4.3 Analysis

4.3.1 Custom task set

The EDF scheduler was tested with the same custom task sets from before.

1. **Task Set 1** ($U < 1$): For Task Set 1, with a utilization of $U = 0.85$, the scheduler handled the tasks efficiently, meeting all deadlines within the hyper-period. The plot in **Figure 4a** shows that all tasks were completed without delays. This confirms that the scheduler operates correctly when the processor is underloaded.
2. **Task Set 2** ($U = 1$): In Task Set 2, where $U = 1$, the scheduler continued to meet all deadlines. The plot in **Figure 4b** shows no missed deadlines, even with the processor fully utilized.
3. **Task Set 3** ($U > 1$): When the utilization exceeds 1, as in Task Set 3, the EDF scheduler struggles to meet deadlines. The plot in **Figure 4c** illustrates missed deadlines due to overloading, which is expected behavior for any scheduler when the processor's capacity is exceeded.

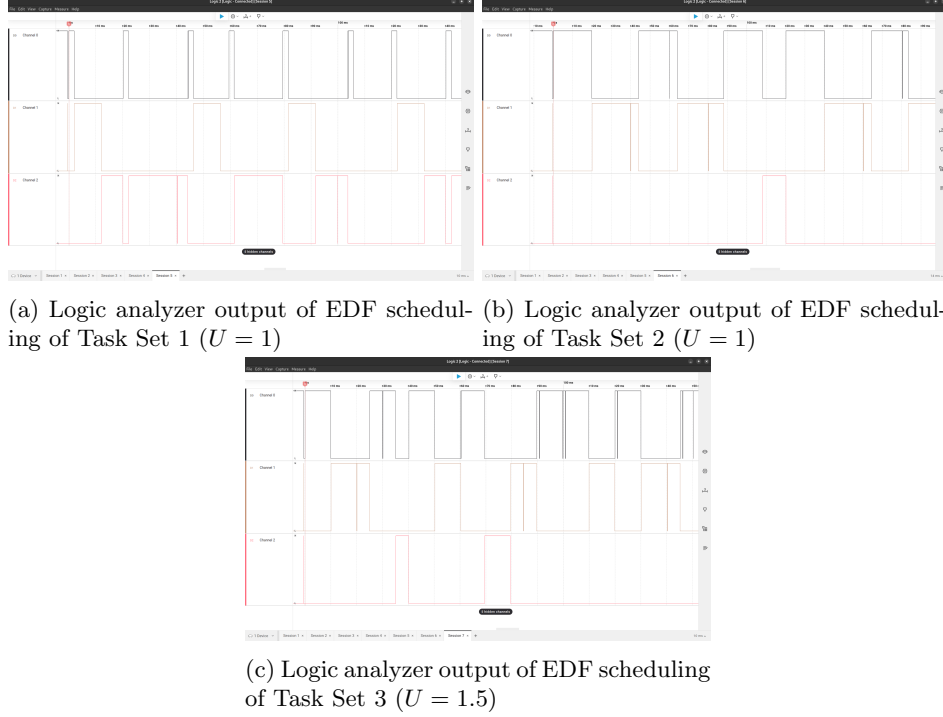


Figure 4: EDF Scheduling Results for Different Task Sets

4.3.2 Overloading

As seen in Task Set 3, the scheduler misses deadlines because the processor is overloaded, leading to missed task completions and potential system instability.

Compared to the RM scheduler, EDF handles overloading differently. While RM has fixed priority levels, EDF allows tasks to be scheduled based on their deadlines, which may sometimes delay higher-priority tasks if they are not the next to meet their deadlines. Thus, in scenarios where $U > 1$, EDF may still attempt to meet deadlines, but it will fail when system load exceeds capacity.

5 Fixed Priority Aperiodic Servers

5.1 Aperiodic Task Analysis

In this section, three jobs will be designed. Each job will have a specific requirement. The hyper-period of the given periodic task set is: $\text{LCM}(20, 40, 60) = 120$ ms. The utilization of the given periodic task set is: $U_{\text{periodic}} = 2/20 + 10/40 + 20/60 = 41/60 = 0.683$. The remaining processor capacity for aperiodic jobs is: $U_{\text{aperiodic}} = 1 - U_{\text{periodic}} = 1 - 41/60 = 19/60 = 0.316$.

1. For the first job, it should never make the periodic task miss its deadline, i.e., $U_{\text{total}} \leq 1$. A possible design:

- (a) Task 1: $C_1 = 5$ ms
- (b) Task 2: $C_2 = 10$ ms
- (c) Task 3: $C_3 = 20$ ms

The total aperiodic execution time is then: $C_{aperiodic} = 35$ ms within the 120 ms hyper-period, i.e. $U_{aperiodic} = 35/120 = 0.292$. This ensures that the total utilization is: $U_{total} = 0.683 + 0.262 = 0.975$.

2. To cause at least one deadline miss for the second job, U_{total} should exceed 1:

- (a) Task 1: $C_1 = 5$ ms
- (b) Task 2: $C_2 = 15$ ms
- (c) Task 3: $C_3 = 25$ ms

Similarly, $C_{aperiodic} = 45$ ms. Thus, $U_{aperiodic} = 45/120 = 0.375$ and $U_{total} = 0.683 + 0.375 = 1.058$. This exceeds 1, indicating that the system is overloaded. Therefore, within the hyper-period, at least one periodic task will miss its deadline.

3. For the third job, the goal is to achieve $U = 1$:

- (a) Task 1: $C_1 = 10$ ms
- (b) Task 2: $C_2 = 10$ ms
- (c) Task 3: $C_3 = 18$ ms

Lastly, $C_{aperiodic} = 38$ ms and $U_{aperiodic} = 38/120 = 0.316$ which leads to $U_{total} = 41/60 + 19/60 = 1$.

5.2 Background Scheduler

In the Background Scheduler, aperiodic tasks only execute when no periodic tasks are ready to run. Periodic tasks maintain their priority according to the scheduling algorithm chosen and only when all periodic tasks are either completed or waiting for their next release, the scheduler assigns execution time to the aperiodic tasks.

5.3 Highest Priority Scheduler

In the Schedule it Immediately (SIE) approach, aperiodic tasks are given top priority at each scheduling point. The moment the scheduler runs, if an aperiodic job is ready, it preempts all periodic tasks. This ensures aperiodic tasks experience minimal latency and short response times, but may cause deadline misses for periodic tasks if the total utilization exceeds 1 or if the aperiodic workload arrives at critical times.

To detect a missed deadline, the scheduler checks a periodic task's completion time against its absolute deadline once the task finishes. If the completion time is greater than the deadline, it declares a deadline miss. Upon detecting a miss, the Orange LED (D7 / `leds[8]`) is turned on to provide an indication of the system's failure to meet timing constraints.

Schedule it Immediately is used when aperiodic tasks need immediate attention, such as in emergency alert systems or robots avoiding collisions, where delays can lead to serious consequences. Background Scheduling, on the other hand, is better for tasks that can wait, such as data logging or file backups, as it ensures that periodic tasks are completed without interruption.

6 Conclusion

In this report, various real-time scheduling algorithms, including Rate Monotonic (RM), Earliest Deadline First (EDF), and hybrid approaches for periodic and aperiodic tasks, were discussed and implemented. A thorough analysis of these schedulers was conducted, and various task sets were designed and flashed to the board and subsequently plotted using the logic analyzer. Unfortunately, the implementations of the schedulers in Part IV did not perform as fully expected.

7 References

References

- [1] Giorgio C. Buttazzo. “Periodic Task Scheduling”. In: *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*. Boston, MA: Springer US, 2011, pp. 79–118. ISBN: 978-1-4614-0676-1. DOI: [10.1007/978-1-4614-0676-1_4](https://doi.org/10.1007/978-1-4614-0676-1_4). URL: https://doi.org/10.1007/978-1-4614-0676-1_4.
- [2] *Course info - Real-time Systems*. <https://cese.pages.ewi.tudelft.nl/real-time-systems/>.
- [3] M.J. Holenderski. *Real-time system overheads : a literature overview*. English. Computer science reports. Technische Universiteit Eindhoven, 2008.
- [4] *Scheduling in Real Time Systems - GeeksforGeeks*. <https://www.geeksforgeeks.org/scheduling-in-real-time-systems/>. May 2023.